Edge cases (where the ¾-word rule breaks) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `token_rule_exceptions_practice.md`: the "~¾ of a word per token" rule works for plain English but breaks badly for code, non-English languages, and structured data, where a single word can cost many tokens. Finding those exceptions on purpose tells you when a rough estimate is fine and when you must count tokens properly. **If not:** a prompt full of JSON or Urdu text would cost and overflow far more than you planned.

Solution

Story: the same cases, plus the reason behind the gap: the tokenizer's vocabulary was trained mostly on English prose, so text unlike that (code, Urdu, JSON) gets split into many more, smaller tokens. **If not:** you'd memorise exceptions instead of being able to predict new ones.

Code: `print('hello world')`

```
naive word-count estimate (2 "words" + punctuation) : ~3 tokens
actual                                              : 5-7 tokens
```

Likely token breakdown: `print`, `(`, `'`, `hello`, `world`, `'`, `)` — roughly one token per syntactic element, because code's punctuation carries meaning the tokenizer's training data treated as worth its own token, unlike a plain-English comma that usually merges into a neighboring word's token.

Non-English: "میں آج بازار جا رہا ہوں" — English equivalent: "I am going to the market today"

```
English-equivalent estimate : ~7 tokens
actual                      : often 15-20+ tokens, sometimes more
```

The mechanism: byte-pair-encoding-style tokenizers build their vocabulary from the most frequent character sequences in a training corpus. A corpus that's mostly English text produces a vocabulary rich in efficient whole-word and common-subword English tokens. A script underrepresented in that training data has few or no such efficient chunks reserved for it, so the tokenizer falls back to much smaller sub-word or even byte-level pieces to represent the same text.

Why this matters practically, beyond trivia: a system prompt full of JSON-schema examples or embedded code tokenizes far worse than its word count suggests, inflating cost on every single call it's part of. A multilingual product priced the same per user regardless of language will quietly cost more per conversation for non-English users — worth knowing before you build a pricing model or a context-budget assumption around English-only testing.